GROUP 9 / IIT GUWAHATI

FACIL

Flexible DRAM Address Mapping

for SoC–PIM LLM Inference

Paper reproduction / Native MapID-aware DRAMsim3 / Batch-aware decode extension

PROJECT TEAM

Sukhjot Singh | **T S S Sreeja** | **Mayank Singh** | **Pankaj**

Indian Institute of Technology Guwahati

What happens after we submit a prompt?

Example: a student asks a course-related question and receives an answer generated token by token.

01 USER INPUT

“This is CS5290. Explain virtual memory in two sentences.”

→

02 TOKEN REPRESENTATIONS

Text → token IDs → vectors

This | is | CS5290 | ...

→

03 TRANSFORMER

Shared learned weights used across all tokens

32 decoder blocks in our Llama3 model

The prompt is available all at once

This | is | CS5290 | ...

The reply is not: each next token depends on earlier generated tokens.

Virtual › memory › lets › ...

This timing difference creates two phases: PREFILL (known prompt) and DECODE (sequential generation).

Text becomes vectors; learned matrices transform them

The token is not multiplied as text—the model processes numerical hidden states.

INPUT TOKEN

“memory”

Token ID → embedding

A learned embedding lookup starts the numerical representation.

HIDDEN STATE

$$x \in \mathbb{R}^{4096}$$

+0.21	−0.47	+0.08	…	+0.35

4,096 numbers per token
(at this representative layer)

LEARNED PROJECTION

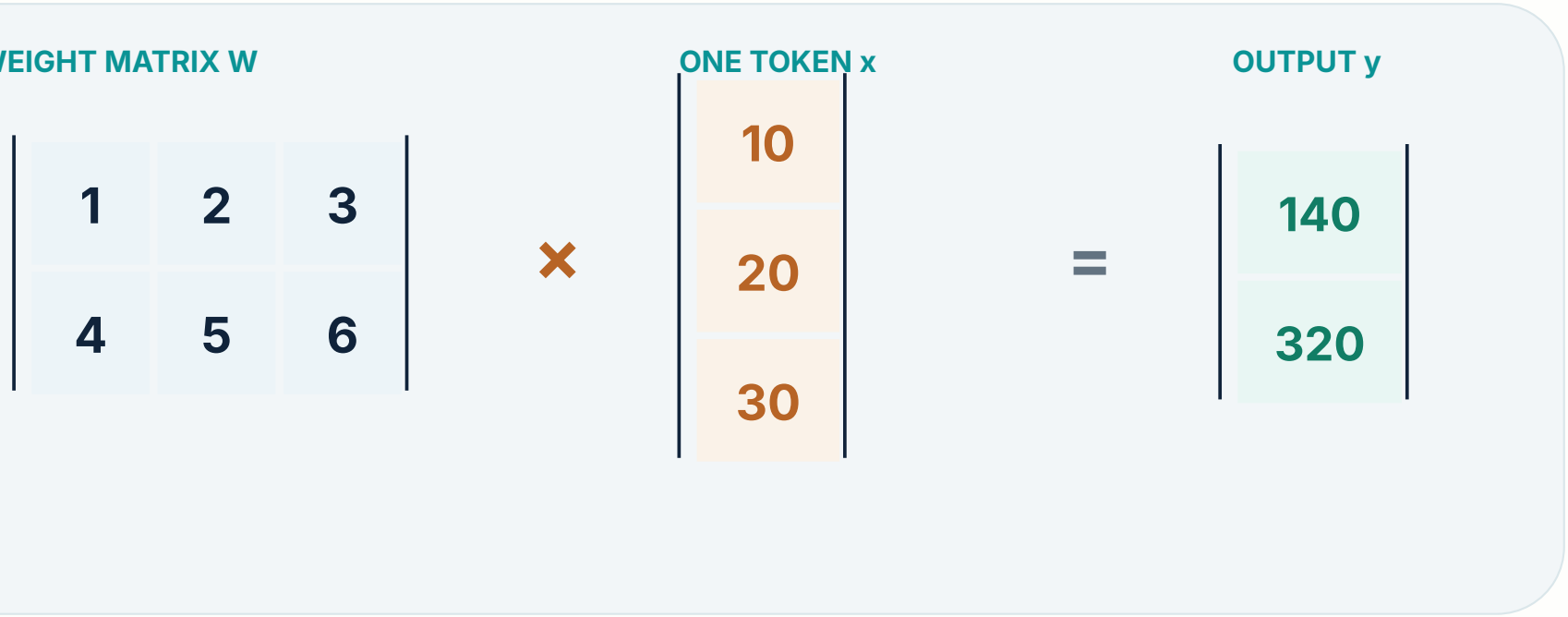
$$y = W x$$

W: trained parameters
x: current activation
y: updated representation

Our workload models Q, K, V, O, gate, up and down projections: 7 per block × 32 blocks = 224

GEMV: the same weight matrix acts on one vector

Matrix–vector multiplication is one of the repeated linear operations during single-sequence decoding.



LLAMA3 PROJECTION

W: 4096 × 4096

x: 4096 × 1

y: 4096 × 1

The same W is used again for the next token.

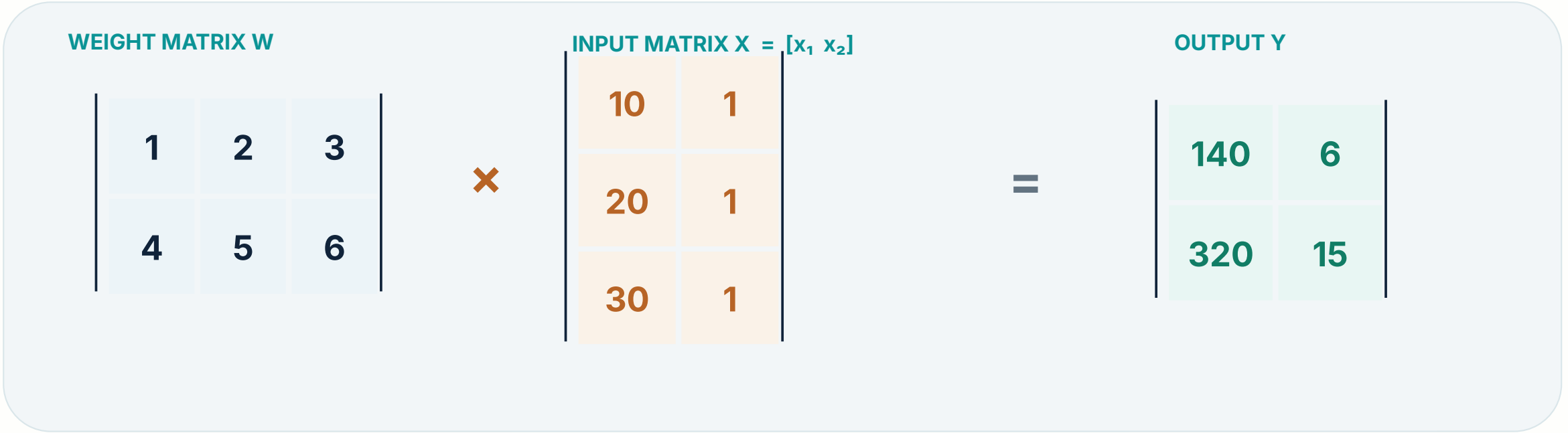
Row 1: 1·10 + 2·20 + 3·30 = 140

Row 2: 4·10 + 5·20 + 6·30 = 320

GEMV = General Matrix–Vector Multiplication

GEMM: group vectors so the same weights can be reused

Now process two independent input vectors together using the identical weight matrix W.



Two separate GEMVs

$W \times x_1$ and $W \times x_2$; repeated weight accesses

One batched GEMM

$W \times [x_1 \ x_2]$; reuse loaded weight tiles across columns

GEMM = General Matrix–Matrix Multiplication

Prefill and decode use the same W, but different input widths

The prompt is known; generated tokens arrive sequentially because the next prediction depends on the previous one.

PREFILL · KNOWN PROMPT

This | is | CS5290 | ...

All prompt positions are available

$$X_{prompt} = [x_1 \; x_2 \; \ldots \; xP]$$

$$W \times X_{prompt} \rightarrow GEMM$$

Many input representations through one projection

DECODE · GENERATED RESPONSE

Virtual › memory › lets › ...

Each next token depends on earlier ones

$$W \times xt \rightarrow GEMV \quad (B = 1)$$

With B independent sequences, decode can form a GEMM

SoC accelerators exploit batched GEMM; PIM is attractive for weight-heavy GEMV. This becomes our execution-policy question.

Why the GEMV/GEMM difference becomes a DRAM problem

The core issue is not just FLOPs: it is how often the processor moves the same learned weights.

Example W = 4096 × 4096 FP16 = 32 MiB of weights

ONE VECTOR / GEMV

≈33.6 million FLOPs

≈1 FLOP per byte of weight data

A large fraction of time can go to moving weights.

32 VECTORS / GEMM

≈1.07 billion FLOPs

≈32 FLOPs per byte of weight data

The same weight tiles serve many input columns.

External weight movement: DRAM weights → SoC / GPU vs. PIM computes near DRAM banks

Different physical weight layouts favor the two engines. FACIL addresses this layout mismatch without storing two copies.

Where the seven linear projections actually occur

Attention uses Q, K, V and O; the gated feed-forward network uses gate, up and down. They are not seven sequential GEMVs.



Q / O	K / V	Gate / Up	FFN Down
4096 × 4096 • 2 matrices	1024 × 4096 • 2 matrices	14336 × 4096 • 2 matrices	4096 × 14336 • 1 matrix

7 matrices / block × 32 blocks = 224 linear-weight projections • Attention arithmetic and nonlinearities are excluded from our timing proxy.

The same weights serve two execution regimes

The same learned weights can feed SoC GEMM or bank-local PIM GEMV.

PREFILL · SOC

GEMM

High parallelism
Conventional contiguous access

DECODE · PIM

GEMV

Memory-bound
Bank-aligned weight placement

But SoC and PIM prefer different ways to place those weights in DRAM.

One output requires one full logical matrix row

A 4096-column FP16 row is 8192 B (8 KiB); our PIM model divides it into four consecutive 2-KiB chunks.

Start with y = W x

Output y_o is the dot product of row $W_{o,:}$ with the 4096-element input x.

One logical row

4096 weights × 2 bytes = 8192 B = 8 KiB.

Chunk width

1024 weights × 2 bytes = 2048 B = 2 KiB.

Logical row $W_{o,:}$ = A0 + A1 + A2 + A3

A0	A1	A2	A3
1024 FP16 values · 2 KiB	1024 FP16 values · 2 KiB	1024 FP16 values · 2 KiB	1024 FP16 values · 2 KiB

$$y_o = (A0 \cdot x[0{:}1024]) + (A1 \cdot x[1024{:}2048]) + (A2 \cdot x[2048{:}3072]) + (A3 \cdot x[3072{:}4096])$$

A logical matrix row (8 KiB) is NOT the same as one modeled DRAM bank row (2 KiB).

The bank stores data; its associated PU performs the computation

In our abstraction, one PU is associated with one bank identified by (channel, rank, bank). DRAM rows reside inside that bank.

DRAM hierarchy

4 channels

↓

2 ranks / channel

↓

16 banks / rank

↓

Many rows / bank

Example: channel 0 · rank 0 · bank 0

DRAM STORAGE ARRAY IN THIS BANK

DRAM row 0 stores A0

DRAM row 1 stores A1

DRAM row 2 stores A2

DRAM row 3 stores A3

Each modeled bank row = 2 KiB

→

PU 0

Computes near this bank

PU 0 does not “contain four DRAM rows.” The rows belong to bank (CH0, RK0, BK0); PU 0 works on data stored there.

The same four chunks can have very different physical homes

Conventional interleaving spreads chunks across banks; PIM benefits from keeping one output row local.

Illustrative conventional mapping

Same logical row: A0 | A1 | A2 | A3

Bank 0	Bank 1	Bank 2	Bank 3
holds A0	holds A1	holds A2	holds A3

Each bank-local PU may produce a partial sum; reduction across PUs depends on the architecture.

PIM-friendly placement: bank (CH0, RK0, BK0)

Four chunks → four DRAM rows within one bank

- **row 0 ← chunk A0**
- **row 1 ← chunk A1**
- **row 2 ← chunk A2**
- **row 3 ← chunk A3**

→ **associated PU 0**

Same 8-KiB logical row. No value in W changes.

The challenge: preserve a normal SoC matrix view while letting PIM use bank-local chunks—without another copy.

Why the baseline has extra read and write traffic

The source weights are PIM-oriented. To use a conventional SoC memory view, the baseline materializes a differently ordered destination.

DRAM region A

PIM-optimized weight placement, existing single copy

→

Read → rearrange → write

Extra DRAM traffic, plus software conversion work not measured here

→

DRAM region B

Conventional SoC-oriented copy used for GEMM

The SoC still reads the weights again to execute GEMM. The conversion reads and writes are additional.

Conventional on-demand baseline

PIM layout → conversion traffic → SoC GEMM. Repeat whenever the baseline needs a conventional representation.

What we would like instead

A single stored weight copy usable by either engine—without converting its layout each time.

The cost of this baseline is extra READ + WRITE traffic. Can address decoding avoid the copy?

BASELINE / METHOD

What we actually measure in the baseline

An SoC-friendly copy is created from PIM-placed weights. The measured quantity is DRAM re-layout traffic.

1 / Weight matrix

K/V: 1024 × 4096
FP16
8 MiB of useful weights

→

2 / Source READ

65,536 requests
128 bytes each

→

3 / Re-arrange

Convert physical layout
CPU work NOT timed here

→

4 / Destination WRITE

65,536 requests
128 bytes each

K/V EXAMPLE · DRAM STAGES

184.560 µs READ + 41.120 µs WRITE

= 225.680 µs of serialized stage-completion upper brackets

NOT PART OF THE MEASUREMENT

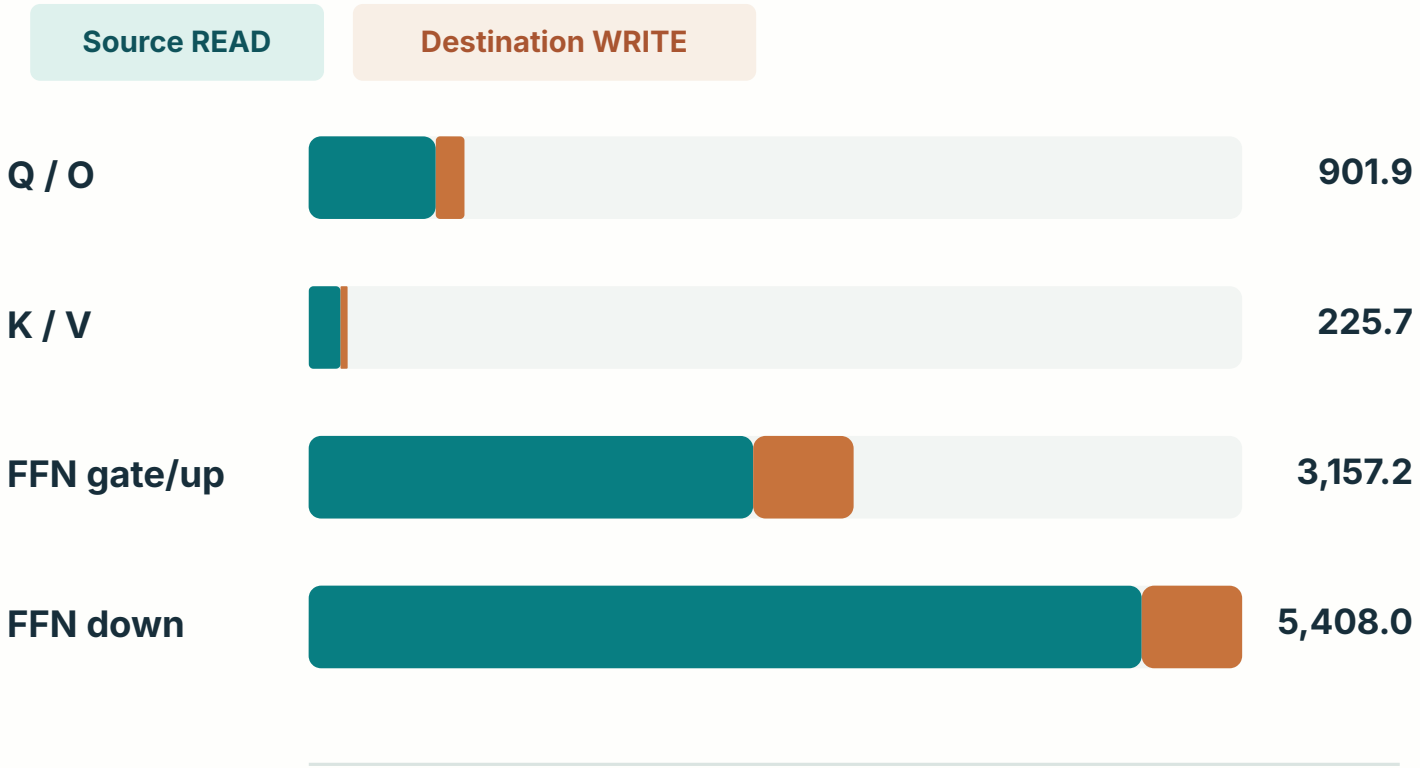
CPU permutation time, real GEMM, cache effects, overlapping work, or complete TTFT.

Baseline re-layout = an additional READ + WRITE before SoC weight access; FACIL avoids this extra copy.

BASELINE / MEASURED DRAM STAGES

How much traffic does one re-layout cost?

Every bar is a DRAMsim3 source READ plus destination WRITE, in µs per matrix.



Stacked length = READ + WRITE; totals shown at bar ends.

STAGE BREAKDOWN (µs)

	READ	WRITE	TOTAL
Q / O	736.0	165.9	**901.9**
K / V	184.6	41.1	**225.7**
FFN gate/up	2,575.3	581.9	**3,157.2**
FFN down	4,826.1	581.9	**5,408.0**

Full 32-block sum: 447.284 ms

WHY FFN DOWN IS DIFFERENT

28-KiB row = 14 chunks; the per-bank page share holds only 8 chunks.

This split corresponds to less favorable mapped READ scheduling here.

From seven matrices per block to the full re-layout proxy

Only the enumerated FP16 linear weights are counted: Q, K, V, O, gate, up and down in 32 decoder blocks.

Weight footprint per decoder block

Q + O	2 × 32 MiB	**64 MiB**
K + V	2 × 8 MiB	**16 MiB**
FFN gate + up	2 × 112 MiB	**224 MiB**
FFN down	1 × 112 MiB	**112 MiB**

Total = 416 MiB / block × 32 = 13 GiB

13 GiB READ + 13 GiB WRITE

26 GiB additional conversion traffic

DRAMsim3 stage sum

32 × [2(0.901920) + 2(0.225680) + 2(3.157210) + 5.407990] ms

447.284 ms

Interpretation: a serialized DRAM traffic-stage proxy, not the time measured to perform software permutation or full LLM inference.

The 204.8 GB/s theoretical external peak is a comparison reference, not a guaranteed rate for all access orders.

Virtual addresses stay conventional; MapID changes DRAM decoding

FACIL selects a DRAM decoding pattern per huge page; our reproduction uses a static page registry.

Virtual addresses

W[0,0]→V, W[0,1]→V+2…

→

Physical address

page base + 21-bit offset

→

Page-selected MapID decoder

channel / rank / bank / row / column

What MapID identifies

It is a decoding scheme, not a PU number: MapID 2 is not Bank 2. Its bit arrangement decides which part of an address selects a DRAM row versus a bank.

Why a 2-MiB huge page matters

2 MiB = 2^{21} bytes: page-offset bits [20:0] can include all channel, rank and bank selection bits. A page can therefore choose its mapping policy.

Selecting a new MapID does not retroactively move data: weights must be populated consistently with their chosen decoding scheme.

MapID 2: four consecutive chunks, same bank, four DRAM rows

With 128-byte requests and a 2-KiB aggregate row, the low bits locate bytes and columns before any bank-select bit changes.

20	19:16	15	14:13	12:11	10:7	6:0
HI	BANK	RK	CH	ROW LOW	COL	OFFSET

OFF [6:0] → 128 B • COL [10:7] → 16 × 128 B = 2 KiB • ROW LOW [12:11] → 4 distinct DRAM rows

Follow successive 2-KiB chunk addresses

A0	**0x0000**	CH0 / RK0 / BK0 / row 0
A1	**0x0800**	CH0 / RK0 / BK0 / row 1
A2	**0x1000**	CH0 / RK0 / BK0 / row 2
A3	**0x1800**	CH0 / RK0 / BK0 / row 3

Next row begins

0x2000 flips channel bit 13, so the next chunk begins at CH1 / RK0 / BK0 / local row 0.

The first logical row stayed entirely in CH0 / RK0 / BK0.

PU 0 is associated with bank CH0 / RK0 / BK0. The 2-KiB data chunks are stored in that bank's DRAM rows.

How MapID steers a memory request

A 2-MiB page selects its MapID; native DRAMsim3 decodes the address and routes the request.

IMPLEMENTED REQUEST PATH

Trace request
A physical address

→

Page registry
MapID 2/3 or conventional

→

Address decode
Channel · rank · bank row · column

→

Channel routing
Select the right controller

→

DRAMsim3
Bank timing and read/write statistics

What the mapping logic guarantees

01 MapID 2 or 3 selects a page-specific DRAM bit layout

02 Decoded channel and bank determine where each request goes

03 Pages without a MapID retain conventional address mapping

Native correctness

411 **Python ↔ C++ address checks passed**

Mixed-page routing: **[72, 8, 8, 8]**

Software-level controller approximation; no OS page-table modification or PIM arithmetic.

FACIL / MATCHED-READ EVIDENCE

A controlled read test reveals a layout trade-off

We replay the same logical read offsets under conventional and FACIL MapID decoding.

Matrix class	128-B reads	Conventional	FACIL MapID
Q / O	262,144	166.389 µs	736.000 µs
K / V	65,536	41.443 µs	184.509 µs
FFN gate / up	917,504	582.301 µs	2,575.300 µs
FFN down	917,504	582.301 µs	4,826.080 µs

4.45×

K/V mapped-scan time relative to conventional

Synthetic DRAM reads do not measure GPU GEMM slowdown.

Mapped – conventional READ time enters only as a sensitivity assumption later.

PREFILL / MODEL BOUNDARY

Building a transparent prefill time estimate

Same 224 linear projections in every case; we change only the modeled layout-related cost.

COMMON MODEL: 7 PROJECTIONS × 32 BLOCKS = 224

G(P) = Σ over 224 projections max(compute time, weight + activation traffic time)

For each M × K weight matrix: compute = 2MKP / 42.5 TFLOP/s; memory = (2MK + 2KP + 2MP) / 204.8 GB/s

BASELINE

G(P) + C

C = 447.284 ms

Extra DRAM READ + WRITE stages

FACIL / IDEAL

G(P)

No added conversion

Assumes no mapped-layout penalty

FACIL / SENSITIVITY

G(P) + Δ

Δ = 308.964 ms

Synthetic read-time increment only

This is a linear-layer TTFT component proxy—not complete TTFT. Attention, output head, sampling, and runtime are excluded.

PREFILL / THREE SCENARIOS

At 16 prompt tokens, what did the model predict?

We compare the same linear work G(16) = 68.567 ms, adding different layout-related components.

BASELINE

515.851 ms

68.567 + 447.284 ms

FACIL / IDEAL

68.567 ms

68.567 ms

FACIL / SENSITIVITY

377.531 ms

68.567 + 308.964 ms

Baseline ÷ FACIL sensitivity

515.851 / 377.531 = 1.366×

138.320 ms saved

= 26.8% less modeled time

WHY IS Δ ONLY THE DIFFERENCE?

G(P) already includes weight access. Adding the entire mapped read time would double-charge those bytes; Δ is only the modeled increment.

C and Δ combine simulator-derived traffic with an analytical GEMM model. These are scenarios, not measured TTFT or guaranteed performance bounds.

PAPER COMPARISON / CLEAR BOUNDARIES

Where our results match FACIL—and where they do not

We reproduce the mapping mechanism and re-layout motivation, not the paper's full system or device-level measurements.

Dimension	Original FACIL paper	Our reduced-scope study
System support	OS / library / controller design	Static page registry + native DRAMsim3 decoder
Timing sources	Device SoC kernels + modeled PIM / DRAM	DRAM traffic replay + analytical GEMM / PIM
Prefill result	2.89× geomean TTFT speedup on Jetson	1.366× projection-only sensitivity ratio
Engine decision	Short-prefill engine selection	Our extension: batch-aware decode choice

WHAT AGREES?

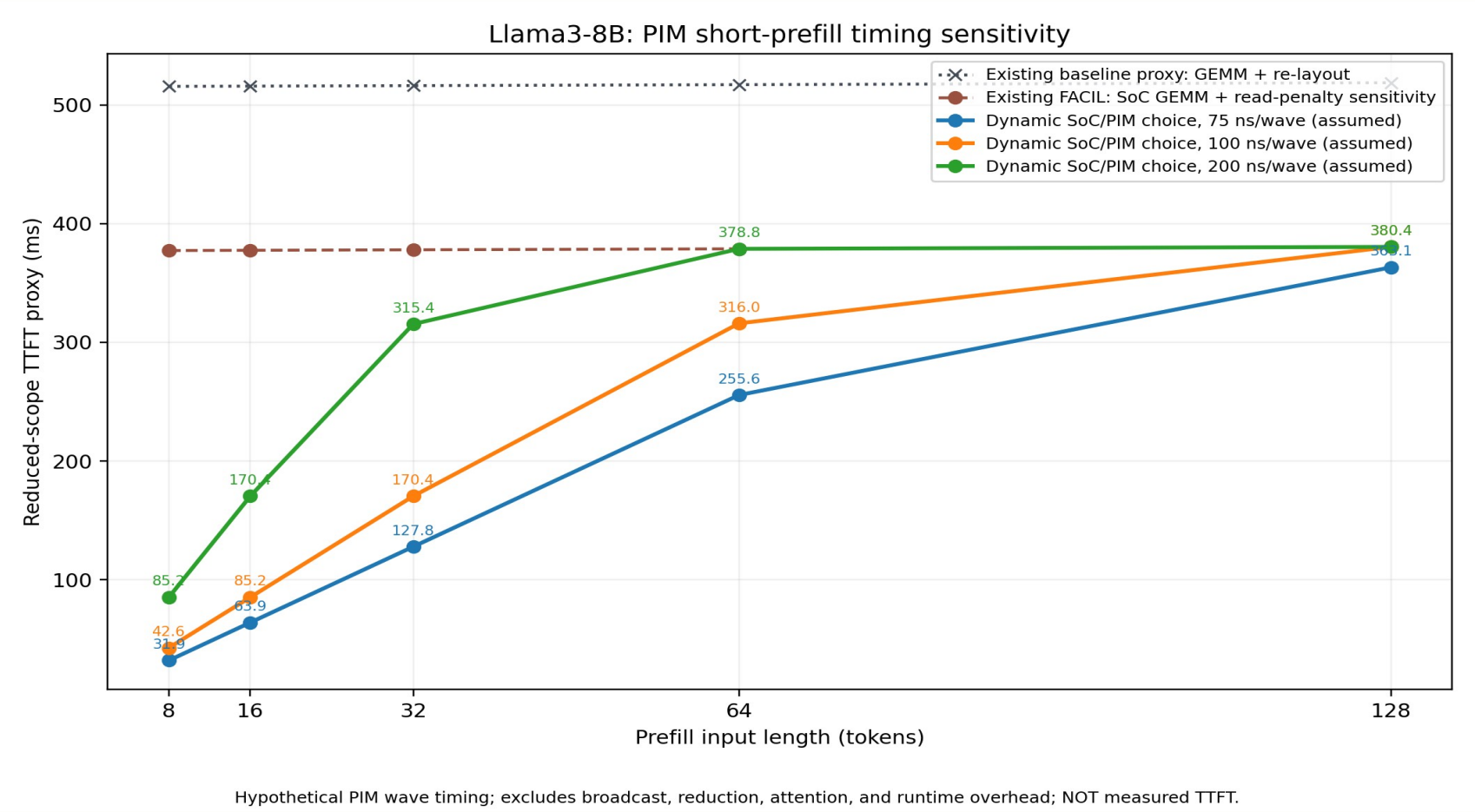
Avoiding repeated re-layout supports the same qualitative benefit of shared weights.

WHY THE SPEEDUPS DIFFER

Full device TTFT and our projection-only proxy measure different work.

Short prefill: the original FACIL engine-choice question

The paper motivates SoC/PIM selection for short prompts; our plot illustrates assumed PIM GEMV timings.



Assumed PIM wave times

75 / 100 / 200 ns

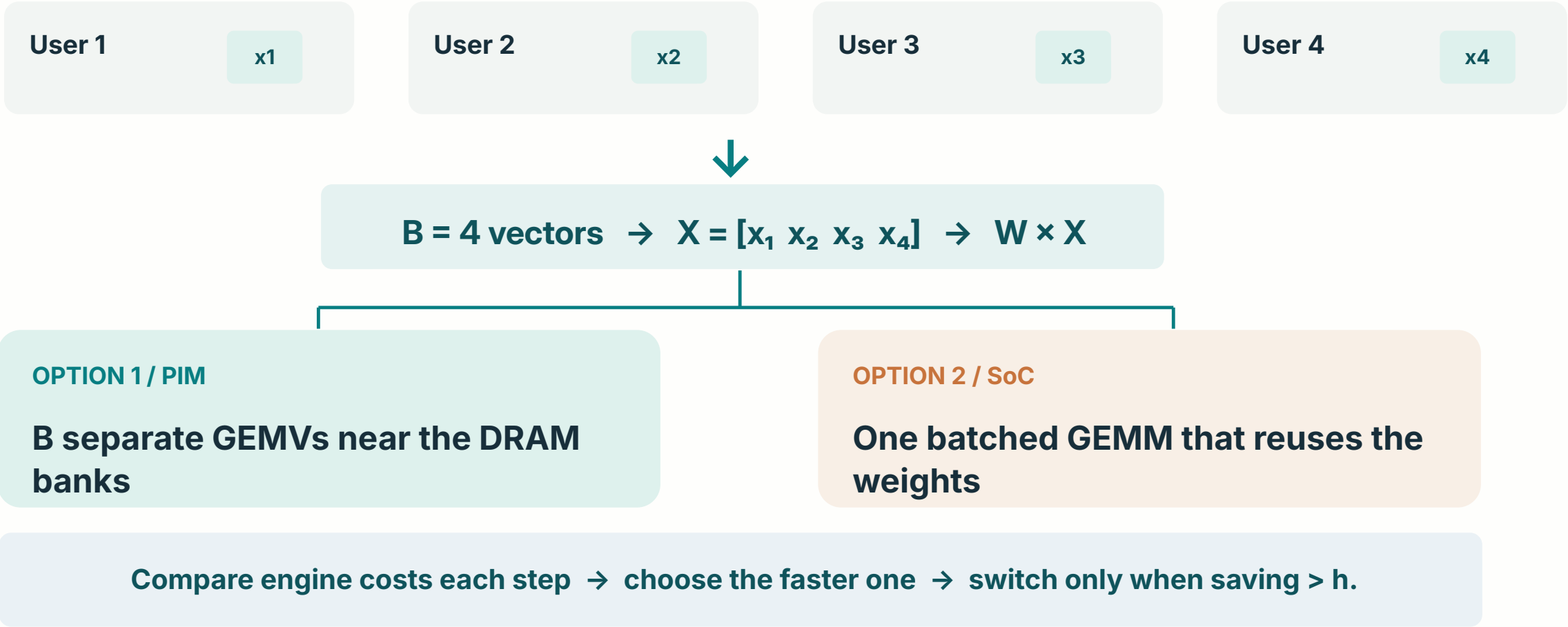
Illustrative PIM model; not measured PIM execution or full TTFT.

Analytical AiM wave counts: Q/O 128, K/V 32, gate/up 448, FFN down 448 per GEMV; never hardware-measured.

BATCH-AWARE DECODE / HOW IT WORKS

Our idea: choose the engine for the current batch

Decode batching means one token from each independent active sequence—not future tokens of one user.



OUR EXTENSION / THE QUESTION

When many users decode together, the best engine can change

Beyond short-prefill engine selection, we study changing batches of independent requests during decode.

SMALL ACTIVE BATCH

B = 1

PIM runs one GEMV close to DRAM weights. There is little shared work to reuse on SoC.

GROWING BATCH

B = 16

More vectors reuse the same weights. A batched SoC GEMM can become attractive.

LARGE ACTIVE BATCH

B = 64

In our model, SoC GEMM wins while repeating separate PIM GEMVs becomes expensive.

Model crossover (η=0.763, ρ=0): first SoC batch is B=17 at 100 ns/wave, or B=5 in the illustrative 20-ms case.

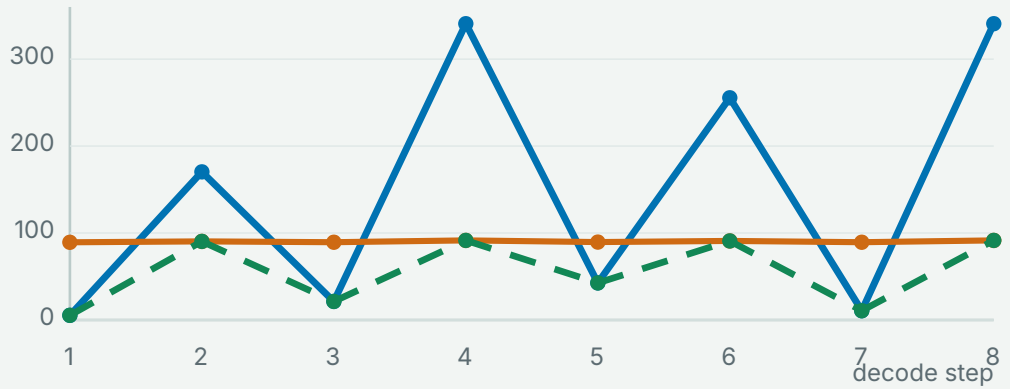
These are analytical break-even points under assumed PIM and SoC parameters, not measured GPU or PIM performance.

DYNAMIC WORKLOAD / MODEL RESULTS

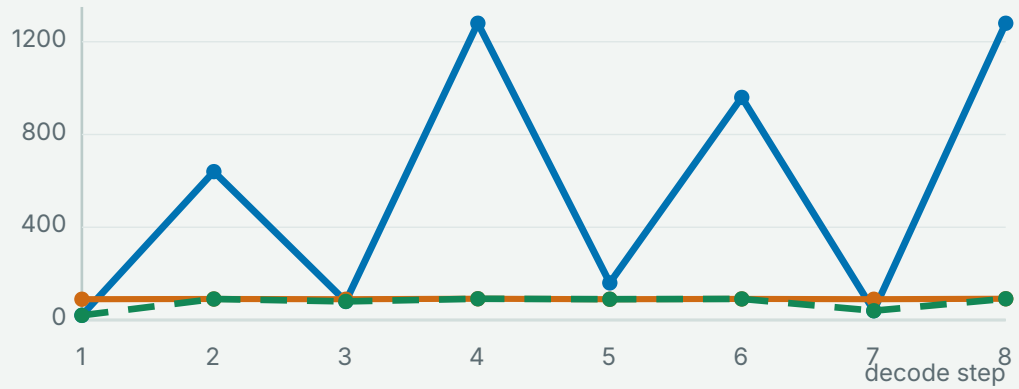
When batch size changes, engine choices change too

Each x-axis point is one decode step. The active batches are: 1 → 32 → 4 → 64 → 8 → 48 → 2 → 64.

100 ns/wave · PIM model · ms



20 ms/token · PIM model · ms



Always PIM Always SoC Best per-step choice (h = 0)

OFFLINE BEST CASE (KNOWS THE FUTURE)

$$T_{schedule} = \Sigma\, T_{chosen}(B_t) + h \times N_{switches}$$

It can avoid a switch if the batch immediately reverses. The first engine choice is free; real online policies do not know future requests.

BREAK-EVEN COST PER SWITCH

h* = max over mixed schedules
(best fixed − mixed at h=0) / switches

Oscillating: 84.037 ms / 69.362 ms

FAIR BASELINE / VERIFIED TWO-COPY TEST

Two copies remove re-layout, but double weight storage

A preplaced PIM layout and a conventional SoC layout let either engine use its preferred weights.

FACIL · ONE SHARED COPY

PIM-compatible weight layout · MapID 2

Both engines access the same stored weights

DUPLICATION · TWO PREPARED COPIES

PIM layout | **SoC layout**

Choose either engine without making a copy at switch time

MATCHED K/V READ SCAN · NATIVE DRAMsim3

184.505 µs FACIL → **41.412 µs** two-copy SoC layout

4.455×

faster SoC scan, at 2× linear-weight storage

13 GiB (13.959 GB) → 26 GiB (27.917 GB) across 224 matrices (calculated footprint, not allocated FP16 bytes). Both PIM layouts verified; GPU GEMM and actual switching time unmeasured.

TAKEAWAYS

What this study establishes

A validated memory-mapping mechanism, a DRAM traffic comparison, and an analytical decode-dispatch extension.

411

native mapping comparisons

Python and C++ decoders agree; per-page routing validated.

447.284 ms

serialized baseline traffic

Calculated DRAM READ+WRITE stage sum across 224 projections.

17 / 5

first SoC-favored batch

Two PIM timing scenarios at $\eta = 0.763$, $\rho = 0$.

THE ARCHITECTURAL TRADE-OFF

FACIL keeps one weight copy; two-copy placement improves our matched SoC read scan but doubles storage.

Measured memory scans, analytic decode costs, and projected TTFT components remain distinct—not end-to-end hardware inference.